



A hybrid approach for fault-tolerance aware load balancing in fog computing

Vijaita Kashyap¹ · Rakesh Ahuja¹ · Ashok Kumar²

Received: 1 September 2023 / Revised: 15 November 2023 / Accepted: 24 November 2023 / Published online: 19 January 2024
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Fog computing has grown in popularity in recent years because to its potential to deliver real-time processing, low latency, and reduce network congestion. However, the implementation of Internet of Things (IoT) enabled smart devices in environments using fog computing may lead to resource limitations and higher computational demands. Load balancing and fault tolerance strategies are necessary to tackle these difficulties for optimal resource usage and system availability. In order to accomplish fault tolerance aware load balancing in fog computing, a hybrid meta-heuristic approach that combines the Modified Harris-Hawks Optimization (MHHO) and Ant Colony Optimization (ACO) is proposed through this paper. The MHHO algorithm is utilized for load balancing, whereas the ACO algorithm is used for fault tolerance. By employing the proposed technique, the load on fog nodes is balanced, the makespan time is minimized, energy consumption and execution costs are minimized, and fault tolerance in fog computing environments is ensured. It is evaluated using a simulation framework built on the iFogSim toolkit. In terms of load balancing, fault tolerance, and other factors, the results of the experiments show that the suggested hybrid algorithm performs better than earlier state-of-the-art methods.

Keywords Fog computing · Fault tolerance · Modified Harris-hawks optimization · Ant colony optimization

1 Introduction

In light of the increasing need for cloud services, the emergence of fog computing has been introduced as a means of extending the principles of cloud computing to the periphery of the network [1, 2]. The aforementioned architecture is a distributed computing system that offers services to end-users located at the network edge, resulting

in decrease in both latency and network congestion [3, 4]. The utilisation of a large number of interconnected devices contributing data to Internet of Things (IoT) applications, such as smart cities, healthcare, and industries, is particularly advantageous in these contexts [5–7]. These devices have the capability to collect and analyse data in a localised manner through the utilisation of fog computing, hence facilitating the process of making real-time decisions [8–10]. Nevertheless, the implementation of fog computing also presents novel obstacles, including the need for efficient load balancing mechanisms and robust fault tolerance strategies.

Load balancing ensures the equitable distribution of workload among nodes, hence mitigating the risk of overburdening certain nodes while underutilizing others [11–13]. In addition, the consideration of fault tolerance and energy economy is vital in load balancing for the system. Faults can be categorised according to several factors, including computing resources, time, and a multitude of concerns. These issues encompass faults in networks, physical components, processors, processes, and service expiration [14]. The issues at hand are effectively tackled through the integration of two optimisation

Rakesh Ahuja and Ashok Kumar have contributed equally to this work.

✉ Vijaita Kashyap
vijaita.kashyap@gmail.com;
vijaita.kashyap@chitkara.edu.in

Rakesh Ahuja
rakesh.ahuja@chitkara.edu.in

Ashok Kumar
ashok.khunger@gmail.com; ashok.27549@lpu.co.in

¹ Chitkara University Institute of Engineering & Technology, Chitkara University, Rajpura, Punjab, India

² School of Computer Application, Lovely Professional University, Phagwara, Punjab, India

approaches, namely MHHO and ACO. The MHHO algorithm is employed to achieve workload distribution that is equitable across all available resources, hence preventing excessive burden on any single node. On the other hand, the ACO algorithm is utilised to ensure fault tolerance by redirecting the workload to alternative nodes in the event of a node failure [15].

The Modified Hawk Hunting Optimisation (MHHO) technique is a novel optimisation approach that has been created based on the hunting behaviour of hawks [16]. The effectiveness of this approach in addressing a range of optimisation problems has been demonstrated [17].

The ACO method is a meta-heuristic optimisation strategy that draws inspiration from the foraging behaviour of ants [18]. The algorithm employed by a group of artificial ants involves the deposition of pheromone trails over the search area in order to determine the optimal solution [19–22]. The proposed methodology is evaluated across many scenarios to illustrate its effectiveness and comparative advantage over alternative algorithms.

1.1 Key contribution

The paper makes the following key contributions:

1. A thorough review of load balancing and fault tolerance in fog computing.
2. A proposed approach and system model for load balancing and fault tolerance in a fog computing environment are discussed.
3. Different scenarios based on fault tolerance are evaluated.
4. A comparison of existing load balancing and fault tolerance strategies with the proposed solution are presented, with several quality characteristics analyzed.
5. A graphical depiction of the findings demonstrates the suggested method's performance compared to state-of-the-art algorithms.

1.2 Paper organization

The remainder of the paper is organized as follows: Sect. 2 provides a summary of the background and relevant work, Sect. 3 describes the proposed technique in detail, Sect. 4 examines the assessment of the suggested approach, and Sect. 5 provides and examines the findings. Finally, Sect. 6 brings the paper to a close.

2 Background and relevant work

For fog computing environments, several load balancing techniques have been suggested, including dynamic, predictive, and reactive load balancing [23, 24]. The dynamic load balancing algorithms monitor and estimate the workload in real-time to balance it across the devices [25, 26]. In order to estimate the workload and allocate resources appropriately, predictive load balancing algorithms employ past data [27, 28]. Reactive load balancing algorithms identify devices that are overloaded and underutilized and change the workload allocation as necessary [8, 29, 30]. These techniques, however, do not take fault tolerance into account and could not be appropriate for fog computing situations where devices may malfunction or stop working.

The capacity of a system to keep operating properly in the face of a breakdown or malfunction is referred to as fault tolerance [31, 32]. Fault tolerance is important in fog computing to keep the system stable and available, especially when edge devices fail because they don't have enough resources or are working in harsh conditions [33, 34]. To achieve fault tolerance, fog computing employs a variety of solutions, including redundancy and load balancing. Duplicating data and processing methods over numerous fog nodes ensures that if one fails, another may take over without affecting the system. This method may be applied at several system levels, including hardware, software, and data [35–37]. To prevent overwhelming any one node and causing a bottleneck, load balancing involves dividing processing work among several fog nodes [38, 39]. By automatically rerouting traffic to a different node in the event of a failing node, load balancing can further improve fault tolerance. Fault detection, checkpointing, and backup servers can all help increase fault tolerance. These systems can detect failures and recover from them automatically, for example, by restarting a malfunctioning process or node or transferring processing duties to another node [40, 41].

An algorithm that solves problems by drawing inspiration from natural occurrences or behaviour is known as a meta-heuristic algorithm. It is a higher-level mathematical framework created to locate good or nearly optimum answers to difficult optimization problems when more standard, precise techniques could be computationally impractical [42]. In order to develop better answers, it usually involves iterative improvement methods and exploring the search space through incremental modification of prospective solutions. These algorithms try to

discover pretty excellent answers in a reasonable amount of time, not necessarily the globally best solution [43].

Basset et al. [44] presented a scheduling technique based on local search strategy through Harris Hawks optimization to reduce energy. The aim of this work is to achieve Quality of Service (QoS) for consumers in Industrial Internet of Things (IIoT) applications. They used a mutation process to improve quality and balance workloads across all Virtual Machines (VMs).

Hussein and Mousa [15] developed a load balancing method that divides IoT jobs across fog nodes using two meta-heuristic algorithms drawn from nature: ACO and Particle Swarm Optimization (PSO). Their primary goals were to reduce communication and response times. The ACO-based methodology successfully distributes the load across the fog nodes and has a faster reaction time when compared to the Round Robin (RR) technique, according to the evaluation of the method.

Wan et al. [6] presented the Energy-aware Load Balancing and Scheduling (ELBS) method for fog environments. To accomplish LB, an energy aware model with the requirement for fog nodes in a cluster has been integrated. For cluster LB, the authors used an enhanced PSO method to find the best solutions. For distributed scheduling, a multi-agent system was also used. Candy packing lines were used to evaluate the ELBS method's effectiveness and confirm its LB and optimum scheduling capabilities in a real-world environment.

Talaat et al. [45] suggested a Load Balancing and Optimization Strategy (LBOS), that uses genetic and reinforcement learning techniques to continuously monitor network traffic and congestion as well as gather information on the workload of each server. This method manages all incoming execution requests and, in order to improve speed, equally distributes them across the available servers for processing. The resource allocator utilized by the authors uses a reinforcement learning approach, and an LB agent is used to choose a fog server to handle incoming requests. They claimed that their algorithm obtained LB up to 85.71% while also being more cost-effective and providing faster reaction time.

Gokul et al. [46] proposed a scheduling method (Min-min and Max-min) adapted from two traditional techniques to enhance resource use. They also proposed a novel meta-heuristic approach that does not suffer from the disadvantages of iterative improvement and allows local searches to escape local optima. Utilizing CloudSim, the meta-heuristic model is assessed and contrasted with PSO and Genetic Algorithm, two already in use methods.

Alarif et al. [31] suggested a fault-tolerant scheduling strategy for fog-cloud IoT-based systems that takes time-sensitive requests into consideration and divides devices into three groups based on the requested service. The

suggested method chooses the best fault-tolerant solution for each request and assesses its success using a variety of performance measures. The system has certain disadvantages, however, including the lack of consideration for network topology, the reliance on a predefined list of devices, and the use of energy.

Oma et al. [47] suggested a fault-tolerant fog computing paradigm for IoT that tackles the issue of fault tolerance in the TBFC model. Fog nodes are designed to be fault-tolerant by incorporating both non-replication and replication FTBFC models. The article evaluated the energy consumption and execution time of non-replication FTBFC models. However, the models have several drawbacks, such as not addressing other fault-tolerant strategies or the models' scalability.

Mohamed et al. [48] focused on the difficulties of obtaining fault tolerance and dependability in fog computing, which may provide a variety of services to serve IoT-based smart city applications. The authors emphasize the relevance of fog computing for smart city applications that require low latency, location awareness, mobility, streaming, management, and real-time support. However, they acknowledged that the fog platform's dependability and fault tolerance are significant challenges that must be addressed.

Sharif et al. [33] proposed ECLB algorithm uses a strategy for classifying different modules and looks at how much energy all Fog Devices (FDs) use. The system then distributes modules among FDs using a Bayesian categorization-based checkpointing and load balancing technique. The authors compared their approach to existing algorithms and found it more effective in terms of energy consumption, cost of execution, network usage, and latency. The suggested ECLB algorithm's actual performance and scalability in a real-world IoT scenario remain unknown. Furthermore, they did not address the possible overhead of implementing the ECLB algorithm on FDs or its influence on the system's overall energy usage.

Rajab et al. [49] introduced a Dynamic Weighted Round Robin (DWRR) approach for scheduling and distributing fog server data based on CPU and memory factors to maximize system performance. Even with high user loads, the approach offers high throughput. The majority of LB approaches disregard Fault Tolerance (FT), which ensures system continuation in the case of a fault. They proposed primary backup replication for FT based on FC dynamic checkpoint intervals. It was triggered dynamically when the value crossed 0.2 of the CPU, reducing costs. The proposed technique reduces execution time when compared to the CPU-independent static checkpoint strategy.

Ramzanpoor et al. [50] implemented a multi-objective fault-tolerant optimization technique for the deployment of IoT applications on fog computing infrastructure. As well

as reducing bandwidth waste and increasing application reliability, the proposed approach will also reduce power consumption and overall latency. A cuckoo search algorithm is used to solve the problem, which is stated as a multi-objective optimization problem. When compared to existing solutions, the proposed methodology significantly improves overall latency and power use. The suggested algorithm's scalability is not explained, and the findings are confined to the given settings. The suggested approach is also not compared to other fault-tolerant optimization techniques.

Simar Preet Singh [51] explains the proposed Fuzzy Golden Eagle Load Balancing algorithm (FGELB) to handle overloaded fog nodes while supporting a high number of IoT devices in fog computing. The proposed system is separated into three phases, which include allocating priority to incoming tasks, rating and planning resources, and managing power. A fuzzy algorithm prioritizes tasks based on deadline time, job size, and specified priorities. As a result, it prioritizes and schedules the resources, and the power management enables or disables them based on their importance. Compared with existing load-balancing systems, FGELB is more energy efficient, has a lower failure rate, computational cost, communication overhead, and faster turnaround time.

This study of the literature discusses several load balancing approaches proposed for fog computing environments, emphasizing their benefits, shortcomings, and efficacy. Meta-heuristic algorithms, multi-agent systems, optimization methodologies, and fault-tolerant scheduling models are among the techniques used. The majority of the approaches aimed to improve QoS, reduce reaction time and communication costs, achieve LB, and increase reliability and fault tolerance. However, several of the approaches have disadvantages, such as not taking network architecture, resource availability, or scalability into account. Furthermore, most strategies evaluate fault tolerance or load balancing using a single methodology. To improve reliability, fault tolerance, and workload distribution in fog computing environments, the proposed hybrid approach, MHHO-ACO, integrates load balancing and fault tolerance techniques.

The Table 1 presents a comprehensive evaluation of the extant literature, revealing an absence of research addressing key aspects and obstacles pertaining to fog computing, such as load balancing and fault tolerance. The approach MHHO-ACO has incorporated several factors and specifically addresses the issues associated with fog computing.

Table 1 Comparison of different parameters for load balancing and fault tolerance

Ref.	MT	Sc	A	EC	O	ECt	S	LB	FT
[15]	X	X	X	X	X	X	X	X	X
[47]	X	X	X	✓	✓	X	X	X	✓
[52]	X	X	X	✓	✓	X	X	✓	X
[53]	X	X	X	X	X	✓	X	X	✓
[49]	X	X	X	X	X	X	X	X	✓
[31]	✓	X	X	X	X	✓	X	X	✓
[14]	X	✓	✓	X	X	X	✓	✓	✓
[33]	X	X	X	✓	X	✓	X	✓	✓
[35]	X	X	X	X	X	X	X	✓	✓
[38]	X	X	X	X	X	X	X	X	✓
[54]	X	X	X	X	X	✓	X	✓	X
[16]	✓	X	X	X	X	X	X	✓	X
[55]	✓	X	X	X	X	X	X	✓	X
[56]	✓	X	X	X	X	✓	X	✓	X
[17]	✓	X	X	X	X	X	X	✓	X
[57]	X	X	X	X	X	X	X	✓	X
[58]	✓	X	X	✓	X	X	X	✓	X
[59]	X	X	X	X	X	✓	X	✓	X
[60]	X	X	X	X	X	X	X	✓	X
MHHO-ACO	✓	✓	✓	✓	✓	✓	✓	✓	✓

MT makespan time, Sc scalability, A availability, EC energy consumption, O overhead, ECt execution cost, S security, LB load balancing, FT fault tolerance

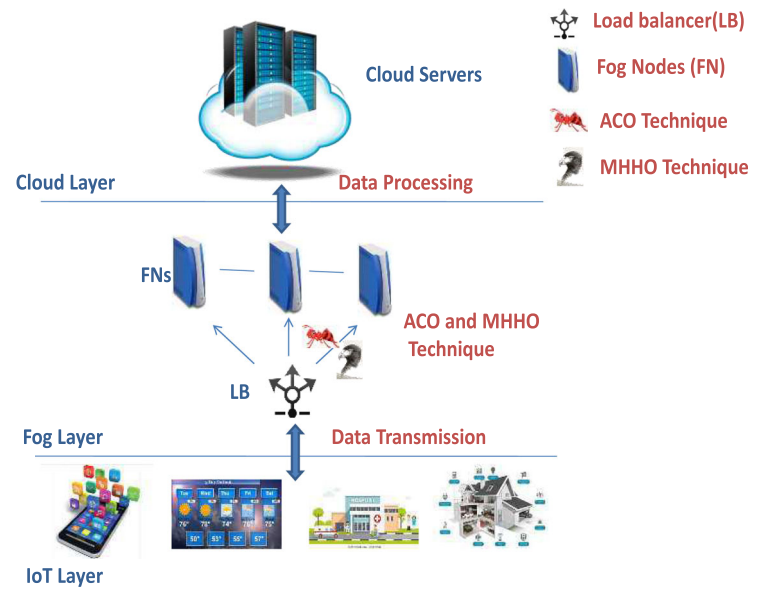
3 Proposed method

This section explains the proposed method, which consists of the following components:

The initial layer in the system is the IoT layer, which includes edge devices such as smart phones, actuators, and sensors. These devices handle data collection and initial processing. The processed data is then transferred to the fog layer, which consists of fog nodes with more computational power and space for storage than edge devices. The fog nodes process the data further and interact with other nodes in the network to share information.

To distribute the burden equally throughout the nodes, the fog nodes employ a hybrid load-balancing algorithm that combines ACO and MHHO. The technique also considers failure tolerance by deploying duplicate instances of the same application across many nodes.

Once the workload is distributed across the fog nodes, the data is transferred to the cloud layer for additional

Fig. 1 Proposed method architecture

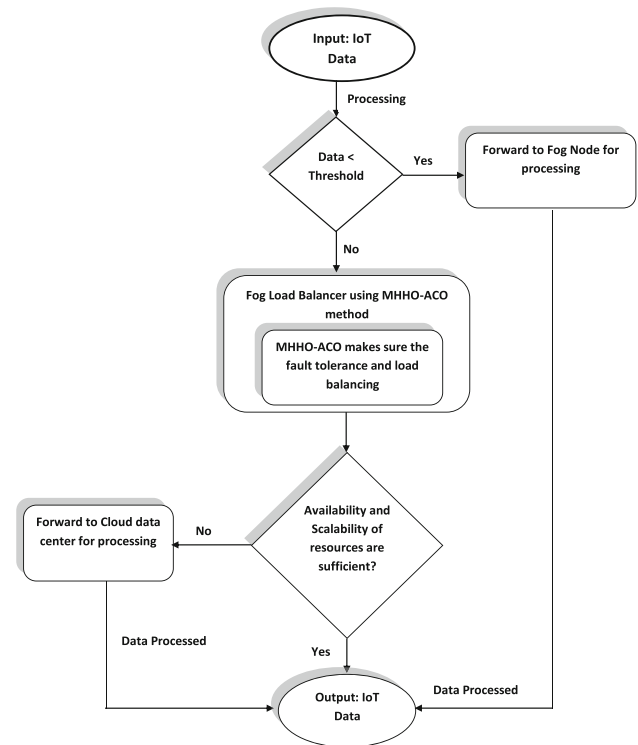
processing and storage. Remote servers and data centres provide scalable computing resources in the cloud layer. The system continually checks the operation of the fog nodes and cloud servers, and if one fails, the burden is diverted to other available nodes or servers to maintain service continuity. Figure 1 shows the architecture of the fault-tolerant and hybrid load balancing method that can distribute the work across the fog layer and make sure that services are always delivered in a fog computing environment.

The flowchart shown in Fig. 2 depicts the step by step process of fault tolerance-aware load balancing approach.

The IoT devices are linked to the edge layer, where the edge processing block assesses whether the data should be transferred to the fog layer or not based on whether it exceeds a specific threshold. The data acquired by IoT devices is sent to the fog layer. The fog layer is composed of several fog nodes that employ the ACO and MHHO algorithms to balance workloads and provide fault tolerance. The data is processed by the fog nodes and sent to the cloud layer. The data is kept in the cloud layer and is then processed by cloud servers.

The MHHO-ACO hybrid algorithm consists of two primary components: MHHO and ACO algorithm. The MHHO algorithm is utilized in the fog computing layer for resource allocation and load balancing, while the ACO is used for fault tolerance by maintaining numerous paths between the fog and cloud computing levels. The steps that describe how both algorithms function are listed below.

- i. Initialization: The method begins by initializing the parameters, such as population size, number of iterations, number of ants, and pheromone parameters.

**Fig. 2** Flowchart of fault tolerance-aware load balancing approach

- ii. Resource Allocation: The MHHO method is employed in this stage to assign the available resources in the fog computing layer. Based on their usage level, the MHHO algorithm attempts to identify the optimal resource allocation to balance the workload across the fog nodes. MHHO's goal function takes into account a number of criteria,

Fig. 3 Input for proposed algorithm

Input:

- F: Set of fog nodes
- T: Set of tasks to be executed
- n: Number of ants
- β : Pheromone factor
- ρ : Pheromone evaporation rate
- MaxIt: Maximum number of iterations
- MaxStagnation: Maximum number of iterations without improvement
- q_0 : Pheromone saturation factor
- α : Scaling factor for local pheromone updates
- p: Probability of applying local pheromone updates
- m: Number of Harris Hawks
- lb: Lower bounds of the search space
- ub: Upper bounds of the search space
- pCrossover: Crossover probability
- pMutation: Mutation probability
- MaxGen: Maximum number of generations
- mu: Mutation rate
- R: Reduction rate
- Keep: Number of elite solutions to keep
- strategy: Selection strategy for mating pool
- fault_tolerance: Fault tolerance mechanism (e.g., redundant deployment, Byzantine fault tolerance)

including energy consumption, execution cost, availability, scalability, overhead, and security.

- iii. Task Allocation: The ACO algorithm is employed in this stage to allocate work across the fog and cloud computing layers depending on priority levels. Using pheromone trails, the ACO algorithm directs the ants to the best path between the fog and cloud levels. By maintaining several paths between the fog nodes and the cloud, ACO also provides fault tolerance. In the event of a path failure or congestion, ACO can switch to the way with the greatest pheromone trail value.
- iv. End: The procedure is completed when a certain number of iterations is reached or the solution converges.

3.1 Algorithms

The algorithms for the proposed fault-tolerance aware load balancing are discussed in this section. Firstly, the algorithm of ACO has been explained that search for the best solution by using the fault tolerance mechanism, next the algorithm of MHHO has been explained that is used for load balancing. The algorithms shown in Figs. 3 and 4 are combined to achieve fault tolerance-aware load balancing in fog computing. The idea is to evenly divide the load across the fog nodes while ensuring that the system

remains fault-tolerant and can continue functioning even in the presence of node failures.

3.1.1 ACO algorithm for fault-tolerance

Algorithm 1 shows the step by step explanation of ACO that find the best solution by applying the fault tolerance method:

Algorithm 1 ACO(MaxIt, n, F, T, MaxStagnation)

```

1: Initialize pheromone trail matrix  $\tau$  to small positive constant values
2: Initialize  $X_{best}$  and  $f_{best}$  to  $+\infty$   $\triangleright$  Best-so-far solution and its objective value
3: Initialize Iter to 1  $\triangleright$  Iteration counter
4: Initialize Stagnation to 0  $\triangleright$  Stagnation counter
5: while Iter  $\leq$  MaxIt and Stagnation  $\leq$  MaxStagnation do
6:   for each ant  $k$  in  $\{1, 2, \dots, n\}$  do
7:     Select a random starting fog node from  $F$ 
8:     for each task  $t$  in  $T$  do
9:       Compute the probability of selecting fog nodes as destinations
10:      Select the next destination node based on the probability distribution
11:      Update local pheromone trail matrix  $\tau_{local}$ 
12:    end for
13:    Update global pheromone trail matrix  $\tau_{global}$ 
14:    Evaluate the solution obtained by the ant
15:    Update  $X_{best}$  and  $f_{best}$  if the current solution is better
16:  end for
17:  Apply fault tolerance mechanism to handle node failures, if any
18:  Apply Modified Harris-Hawks Optimization algorithm to generate new solutions
19:  Update Iter
20:  Iter = Iter + 1
21: end while
22: Return Best_Solution

```

Fig. 4 Algorithm of the proposed approach**Output:**

- Xbest: Best solution found
- fbest: Best objective function value found

1. Initialize the pheromone trail matrix τ to a small positive constant value.
 2. Initialize the best-so-far solution Xbest and its corresponding objective function value fbest to infinity using **equation (1)**.
 3. Initialize the iteration counter Iter to 1.
 4. Initialize the stagnation counter Stagnation to 0.
 5. Repeat until the maximum number of iterations MaxIt is reached or the stagnation counter Stagnation exceeds MaxStagnation:
 - (a) For each ant $k = 1, \dots, n$, do the following:
 - (i) Randomly select a starting fog node from F.
 - (ii) For each task t in T, do the following:
 - (1) Compute the probability of selecting each fog node as the next destination using the pheromone trail matrix τ and the heuristic information η using **equation (2)**.
 - (2) Select the next destination node based on the probability distribution **equation (3)**.
 - (3) Update the local pheromone trail matrix τ_{local} using **equation (4)**.
 - (iii) Update the global pheromone trail matrix τ_{global} using **equation (5)**.
 - (iv) Evaluate the solution obtained by the ant.
 - (v) Update the best-so-far solution Xbest and its corresponding objective function value fbest if the current solution is better.
 - (b) Apply fault tolerance mechanism to handle node failures, if any.
 - (c) Apply Modified Harris-Hawks Optimization algorithm on the current population to generate new solutions.
 - (i) Compute the fitness values of the Harris Hawks using **equation (6)**.
 - (ii) Sort the Harris Hawks based on their fitness values in ascending order.
 - (iii) Update the global best solution Xbest using equation..
 - (iv) For each Harris Hawk $i = 1, \dots, m$, do the following:
 - (1) Generate a new solution using either a crossover or a mutation operation with probabilities pCrossover and pMutation, respectively.
 - (2) Apply a reduction operator on the new solution with probability μ .
 - (3) Evaluate the fitness of the new solution.
 - (4) If the fitness of the new solution is better than that of the current Harris Hawk i , replace the current Harris Hawk i with the new solution.
 - (v) Update the pheromone trail matrix τ using **equation (7)**.
 - (d) Update the iteration counter Iter
 Iter = Iter + 1
 End of the while loop.
 - (vi) Return the best solution obtained during the optimization process.
- Return Best_Solution

The time complexity of the algorithm can be evaluated as follows:

1. **Initialization:** The pheromone trail matrix and certain variables are initialised in this stage. The amount of time it takes is $O(n)$, where n is the number of ants.
2. **While Loop:** The main loop runs for a maximum of MaxIt times. Every time its updated, it need to go through all the ants and do a number of things for each one. The time complexity of each iteration can be evaluated as:

- The steps for each job in T for each ant. If there are m tasks, each ant takes $O(m)$ time to do operations that are related to those tasks.
- In the worst case, updating pheromone matrices requires updating edges, the magnitude of which can be $O(n^2)$, contingent on the quantity of edges.
- In order to assess the performance of each ant, it may be necessary to compute the objective value, which, contingent upon the particular problem at hand, may not exceed $O(m)$.

Consequently, the per-iteration time complexity of the ACO component is approximately $O(m + n^2)$.

3.1.2 MHHO algorithm for load balancing

Algorithm 2 shows the step by step explanation of MHHO for load balancing:

Algorithm 2 MHHO(PopSize, MaxIt, pCrossover, pMutation, mu)

```

1: Initialize a population of Harris Hawks with size PopSize
2: Initialize pheromone trail matrix  $\tau$  to small positive constant values
3: while Iter  $\leq$  MaxIt do
4:   Compute the fitness values of Harris Hawks
5:   Sort Harris Hawks based on fitness values in ascending order
6:   Update global best solution  $X_{best}$ 
7:   for each Harris Hawk i in  $\{1, 2, \dots, PopSize\}$  do
8:     Generate a new solution using crossover or mutation with probabilities
       pCrossover and pMutation
9:   Apply a reduction operator to the new solution with probability mu
10:  Evaluate the fitness of the new solution
11:  if the fitness of the new solution is better then
12:    Replace the current Harris Hawk i with the new solution
13:  end if
14: end for
15: Update the pheromone trail matrix  $\tau$ 
16: Increment Iter by 1
17: end while
18: Return BestSolution

```

The time complexity of the MHHO algorithm depends on the population size (PopSize), the number of iterations (MaxIt), and the complexity of operations within each iteration.

1. **Initialization:** The complexity of this phase is $O(\text{PopSize})$
2. **While Loop (MaxIt iterations):**
 - Sorting the population based on fitness: $O(\text{PopSize} \cdot \log(\text{PopSize}))$ in the worst case (using a comparison-based sorting algorithm).
 - Operations within each iteration (for each Harris Hawk):
 - Generating a new solution using crossover or mutation: Depends on the specific method, let's assume $O(m)$ for both.
 - Applying a reduction operator: Depends on the specific method, let's assume $O(m)$.
 - Evaluating the fitness: Depends on the specific problem, let's assume $O(m)$.
3. **Overall Time Complexity:**

$$O(\text{MaxIt} \cdot (\text{PopSize} \cdot \log(\text{PopSize}) + m))$$

3.2 Hybrid algorithm input (ACO and MHHO)

The pheromone trail matrix is initialized at the start of the process, along with the initial population of potential

solutions. The ACO component then selects the best option from the current population, updating the pheromone trail matrix. The weights given to each job are then updated using the MHHO component based on the chosen solution and the fog node performance.

This process of choosing the optimal solution, updating the pheromone trail matrix, and changing the weights given to each job is repeated for a predetermined number of repetitions. The method checks at the end of each loop to see if any nodes have failed and redistributes the burden accordingly to guarantee that the system remains fault-tolerant.

The equation for the objective function:

$$f(x) = \alpha \cdot Av(x) + \beta \cdot Sc(x) + \gamma \cdot O(x) + \delta \cdot C(x) + \epsilon \cdot E(x) + \zeta \cdot M(x) + \eta \cdot T(x) \quad (1)$$

where $Av(x)$ represents availability, $Sc(x)$ represents scalability, $O(x)$ represents overhead, $C(x)$ represents security, $E(x)$ represents execution cost, $M(x)$ represents energy consumption, and $T(x)$ represents makespan time.

Heuristic information equation:

$$\eta(i, j) = \frac{1}{d(i, j)} \quad (2)$$

where i, j and $d(i, j)$ are the nodes' Euclidean distances.

The equation for the local pheromone update:

$$\tau_{local}(i, j) = (1 - \alpha) \cdot \tau_{local}(i, j) + \alpha \cdot \tau_0 \quad (3)$$

where $\tau_{local}(i, j)$ represent pheromone trail value for edge (i, j) updated by the ant k , τ_0 is a small positive constant value, and α is a scaling factor for the local pheromone updates.

The equation for the global pheromone update:

$$\tau_{global}(i, j) = (1 - \rho) \cdot \tau_{global}(i, j) + \rho \cdot \Delta\tau_{global}(i, j) \quad (4)$$

where edge's global pheromone trail value is $\tau_{global}(i, j)$, where ρ is the pheromone evaporation rate and the amount of pheromone that the ant k deposited on the edge (i, j) is represented by $\Delta\tau_{global}(i, j)$.

The equation for the fitness function in MHHO:

$$fitness(i) = \frac{1}{1 + f(i)} \quad (5)$$

where the value of the objective function for the i th solution is $f(i)$.

The equation for the position update in MHHO:

$$x(i) = x(i) + r1 \cdot (x(best) - x(i)) + r2 \cdot (x(r) - x(s)) \quad (6)$$

where $x(i)$ represents the location of the i th Harris hawk, $x(best)$ represents the location of the best solution globally, $x(r)$ represents the location of a randomly chosen Harris

hawk, $x(s)$ represents the location of the worst Harris hawk, and $r1$ and $r2$ represent random coefficients.

The equation for the pheromone update in MHHO-ACO:

$$\tau(i, j) = (1 - R) \cdot \tau(i, j) + R \cdot \text{fitness}(i) \cdot \text{fitness}(j) \quad (7)$$

where $\text{fitness}(i)$ and $\text{fitness}(j)$ are the fitness values of the i th and j th Harris hawks, respectively, and $\tau(i, j)$ is the pheromone trail value for edge (i, j) .

3.3 Time complexity of hybrid ACO-MHHO

The time complexity of the hybrid algorithm can be expressed as the sum of the time complexities of its components ACO and MHHO:

$$\begin{aligned} \text{Total Time Complexity} &= O(\text{MaxIt} \cdot (m + n^2)) \\ &+ O(\text{MaxGen} \cdot (\text{PopSize} \cdot \log(\text{PopSize}) + m)) \end{aligned}$$

The overall time complexity is dominated by the component with the highest time complexity, which can vary depending on the problem and parameter settings. Additionally, the evaluation of the fitness function, which is problem-dependent, may significantly impact the running time.

4 Experimental evaluation

To evaluate the efficacy of the proposed hybrid algorithm, a number of experiments were performed using the iFog-Sim simulation toolkit. The simulation was run on an HP laptop with the following system specifications: CPU (Intel x64-based dual-core 2.6 GHz) with 6 GB of RAM and the Windows 10 64-bit operating system. Consider a fog computing system with a single cloud, many fog nodes, and a collection of IoT devices generating a variable number of jobs. For fault tolerance-conscious load balancing in fog computing, the performance of this suggested hybrid algorithm was compared to the standard ACO, PSO, HHO, and Firefly Algorithm (FA). The following are the assessment metrics:

1. **Makespan time:** The amount of time it takes to finish all the jobs in a certain situation is called “makespan time.” It is found by adding up the times it takes to complete all the jobs.

$$MT = \sum_{i=1}^n T_i \quad (8)$$

where

n is the total number of tasks.

T_i is the execution time of task i .

2. **Execution cost:** The execution cost is the total amount of money that was spent on all jobs. It is found by increasing the time it takes to do each job by how much it costs.

$$EC = \sum_{i=1}^n (T_i \times CR_i) \quad (9)$$

where

n is the total number of tasks. T_i is the execution time of task i . CR_i is the cost rate of task i .

3. **Energy consumption:** The total amount of energy used to complete all jobs is shown by energy usage. It is found by multiplying the rate at which energy is used by the time it takes to complete each job.

$$ECt = \sum_{i=1}^n (T_i \times ECR_i) \quad (10)$$

where

n is the total number of tasks.

T_i is the execution time of task i .

ECR_i is the energy consumption rate of task i .

The experiments have been carried out with the settings shown in Table 2:

The experiments were conducted for three different fault scenarios:

1. **No fault scenario:** In this scenario, all fog nodes were expected to be operational with no failures.
2. **Partial fault scenario:** In this scenario, it was believed that a certain number of fog nodes were faulty and unable to execute their functions.
3. **Full fault scenario:** The assumption in this scenario is that all of the fog nodes were faulty and could not complete their jobs.

The efficacy of various algorithms is compared in Table 3, Table 4, and Table 5 under various fault scenarios, including no fault, partial fault, and full fault conditions. The proposed method MHHO-ACO achieves the lowest makespan time (92.22 s), execution cost (470.21 dollars), and energy consumption (116.01 joules) in Table 3, which represents a no-fault scenario. These results demonstrate the algorithm’s efficacy and cost-effectiveness under optimal operating conditions.

In Table 4, a partial-fault scenario simulating a more difficult scenario is examined. With the lowest makespan time (117.44 s), execution cost (558.91 dollars), and energy consumption (150.41 joules), MHHO-ACO continues to excel. This demonstrates the algorithm’s resilience and adaptability in the face of a certain level of system errors or disturbances.

Table 2 Experimental settings

No. of IoT devices	No. of fog nodes	Total no. of tasks	Max. No. of iterations	Population size
100	5	1000	500	50
200	10	2000	500	50
300	15	3000	500	50
400	20	4000	500	50

Table 3 Comparison of the performance of different algorithms in a no-fault scenario

Algorithm	Makespan time	Execution cost	Energy consumption
ACO	99.70	490.31	126.21
HHO	114.05	523.42	148.72
PSO	120.12	567.21	163.50
FA	107.42	525.16	140.82
MHHO-ACO	92.22	470.21	116.01

Table 4 Comparison of the performance of different algorithms in a partial-fault scenario

Algorithm	Makespan time	Execution cost	Energy consumption
ACO	129.81	615.52	175.00
HHO	157.41	732.85	224.11
PSO	166.70	782.41	237.62
FA	142.10	682.43	200.44
MHHO-ACO	117.44	558.91	150.41

Table 5 Comparison of the performance of different algorithms in a full-fault scenario

Algorithm	Makespan time	Execution cost	Energy consumption
ACO	212.48	1406.60	6.55e6
HHO	221.71	1467.66	6.50e6
PSO	210.32	1401.49	6.66e6
FA	209.44	1399.14	6.52e6
MHHO-ACO	199.63	1343.41	6.25e6

Table 6 Comparison of the proposed algorithm with other algorithms in terms of several metrics

Algorithm	MT	ECt	EC	A	Sc	O	S
ACO [15]	381.31	1624.26	199.23	M	H	H	L
PSO [60]	401.80	1790.92	225.43	M	H	H	L
HHO [58]	365.61	1490.13	191.81	H	M	L	H
FA [59]	357.45	1421.91	175.70	H	M	L	H
MHHO-ACO	338.24	1235.70	165.23	H	H	L	H

MT makespan time, *ECt* execution cost, *EC* energy consumption, *A* availability, *Sc* scalability, *O* overhead, *S* security

The algorithms are evaluated in Table 5 under a full-fault scenario, in which the system encounters significant disruptions. MHHO-ACO algorithm has the lowest makespan time (199.63 s), lowest execution cost (1343.41 dollars), and relatively low energy consumption (6.25e25

joules) even under these difficult conditions. These results demonstrate the algorithm's robustness and efficacy in managing complex malfunction scenarios, making it an excellent option for mission-critical applications.

The final comparison is presented in Table 6, which includes metrics such as Availability, Scalability, Overhead, and Security. In each of these tables, MHHO-ACO consistently demonstrates its ability to manage a variety of defect scenarios, cost-effectiveness, energy efficiency, high availability, scalability, low overhead, and strong security. By enlarging the data, a different scenario was once more chosen. The metrics are assessed for the entire failure scenario, which includes 500 IoT devices, 25 fog nodes, and 5000 tasks. The values are calculated by averaging the results of 10 separate runs of each program. The parameters are measured in the following units: makespan time in seconds, execution cost in dollars, and energy usage in joules. In this case, the calculation of the availability,

scalability, overhead, and security that are rated as High (H), Medium (M), and Low (L) is also considered..

4.1 Fault tolerance metrics

Some of the existing fault tolerance metrics are discussed below:

1. **Makespan Time:** It implies the total length of time required to complete a job or set of tasks. The goal is to minimize the makespan time by allocating jobs to fog nodes as efficiently as possible.
2. **Execution cost:** The cost of carrying out a set of activities on a given set of resources. It covers the cost of utilizing resources like computer power and memory, as well as other overhead charges. The objective is to reduce execution costs by distributing workloads to nodes with low resource utilization and energy consumption.
3. **Energy consumption:** This value represents the amount of energy that fog nodes use while operating. The objective is to reduce energy usage by spreading jobs across fog nodes as effectively as possible.
4. **Availability:** It reflects the availability of the network's fog nodes. The objective is to guarantee that tasks are allocated to fog nodes that are highly available in order to reduce the likelihood of task failure.

$$A = \frac{T_{\text{uptime}}}{T_{\text{total}}} \quad (11)$$

where

T_{uptime} is the total time the system is operational.

T_{total} is the total time over a given period.

5. **Scalability:** It refers to the system's capacity to manage increased demands. The objective is to make sure that jobs are assigned to fog nodes with enough resources to manage the burden.

$$Sc = \frac{T_{\text{performance}_2}}{T_{\text{performance}_1}} \quad (12)$$

where

$T_{\text{performance}_1}$ is the system's performance under one load.

$T_{\text{performance}_2}$ is the system's performance under a different load.

6. **Overhead:** It denotes the overhead related to job assignment and data transport across fog nodes. The objective is to reduce overhead by distributing jobs to fog nodes as effectively as possible.

$$O = T_{\text{with_overhead}} - T_{\text{without_overhead}} \quad (13)$$

where

$T_{\text{with_overhead}}$ is the time or resources required with overhead.

$T_{\text{without_overhead}}$ is the time or resources required without overhead.

7. **Security:** It symbolizes the safety of the fog nodes and the data they process. The objective is to guarantee that jobs are assigned to fog nodes that have high levels of security and privacy.

5 Comparative analysis

Table 7 presents a comparison of the suggested method with other approaches in terms of the technology employed, parameters utilised, results obtained, and limitations identified. The research conducted by other scholars has demonstrated a lack of consideration for key factors, including load balancing and fault tolerance.

6 Result and discussion

The following are essential aspects to emphasise concerning the significance of the MHHO-ACO hybrid algorithm in attaining load balancing and fault tolerance inside a fog computing environment:

1. **Efficient load balancing:**

The MHHO-ACO hybrid algorithm demonstrates superior performance in effectively and efficiently allocating computational workloads among fog nodes. The optimisation of resource allocation is achieved by taking into account multiple characteristics, including execution cost, energy consumption, availability, scalability, overhead, and security. The use of load balancing guarantees the equitable distribution of duties among fog nodes, hence mitigating the risk of any individual node becoming overwhelmed and experiencing resource depletion, which could lead to a decline in system performance.

2. **Minimization of makespan time:**

The technique has been specifically developed with the objective of minimising the makespan time, a critical factor for applications that require real-time processing and time-sensitive operations. By minimising the duration of the makespan, it guarantees the timely completion of duties, hence enhancing the overall service quality.

3. **Enhanced fault tolerance:**

The MHHO-ACO hybrid algorithm incorporates fault tolerance techniques inside the fog computing layer. The system possesses the capability to immediately identify and address instances of node failures or congestion, hence ensuring the timely rerouting of workloads to operational nodes. The fault tolerance

Table 7 Comparative analysis

Authors	Technology used	Parameters considered	Results	Limitations
[15]	ACO, PSO	Communication cost, Response time	ACO-based scheduler improves response times	Comparison limited to RR algorithm
[60]	Particle Swarm Optimization (PSO)	Task waiting Time, Latency, Bandwidth, QoE	Reduces task waiting time, latency, etc	Does not specify the context or scope of fog
[58]	Three-Objective Harris Hawks Alg	Makespan Time, Energy Consumption, Reliability	Outperforms WOA, FA, PSO	Reliability is not explained in detail
[59]	Firefly Algorithm	Load Balancing, Cost Reduction	Utilizes Firefly Alg. for Load Balancing	Less storage, increased security compared to Cloud
MHHO-ACO (Proposed Algorithm)	MHHO, ACO	Makespan Time, Execution Cost, Energy Consumption, Availability, Scalability, Overhead, Security	Achieves low makespan time, execution cost, and energy consumption. Offers high availability and security	Comparison limited to HHO, PSO, ACO, and FA algorithms

method serves to augment the resilience and availability of the system.

4. Optimised resource utilisation:

The method optimally utilises resources by dynamically reallocating them according to the system's demands and the incoming workload. The implementation of this approach results in the reduction of resource wastage and the mitigation of costs related to over-provisioning.

5. The ability to adapt to changing workloads:

The hybrid approach demonstrates a high degree of adaptability to variations in workload patterns and the integration of new Internet of Things (IoT) devices without disruption. The system is capable of dynamically reconfiguring itself in order to effectively manage higher levels of traffic while maintaining optimal performance.

6. Robust scalability:

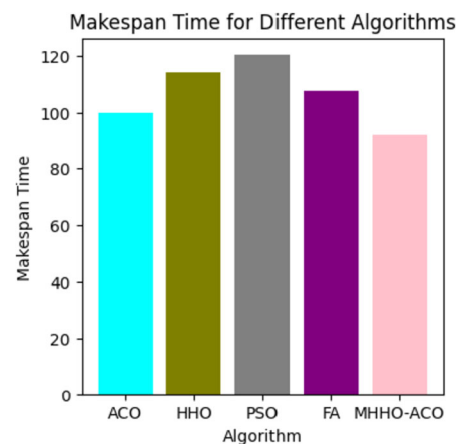
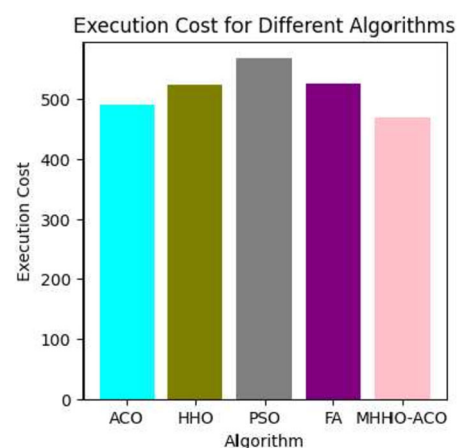
The technique facilitates the scalability of fog computing environments by enabling the seamless integration of additional fog nodes and Internet of Things (IoT) devices. The MHHO-ACO hybrid algorithm ensures a proportional allocation of work as the system expands.

7. The Preservation of Security and Privacy:

The algorithm effectively mitigates security and privacy problems by incorporating security as a key parameter inside its optimisation process. Sensitive tasks can be sent to fog nodes that offer higher levels of security, thereby protecting data and guaranteeing adherence to privacy requirements.

8. Enhancement of Performance Metrics:

The utilisation of the MHHO-ACO hybrid algorithm leads to notable improvements in various performance

**Fig. 5** Comparison of makespan time in a No-fault Scenario**Fig. 6** Comparison of execution cost in a No-fault Scenario

parameters, including execution time, energy consumption, dependability, scalability, overhead, and

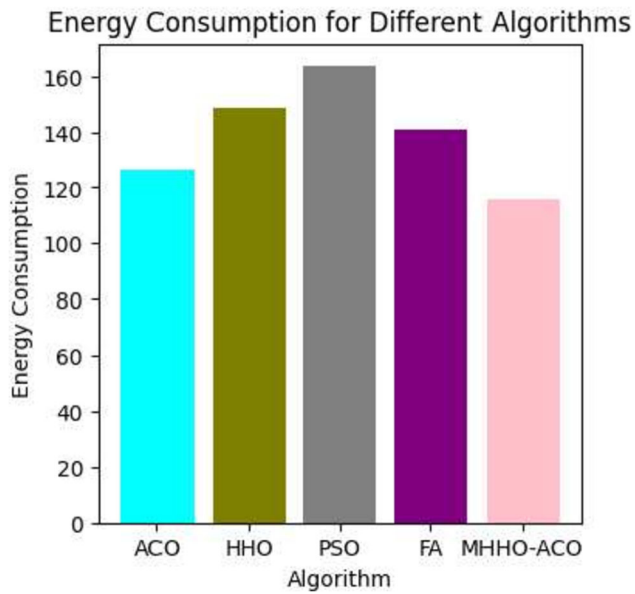


Fig. 7 Comparison of energy consumption in a No-fault Scenario

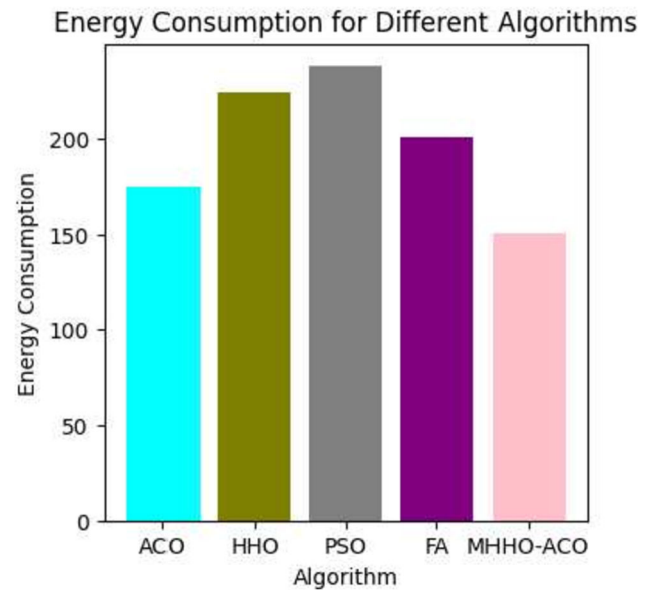


Fig. 10 Comparison of energy consumption in a Partial-fault Scenario

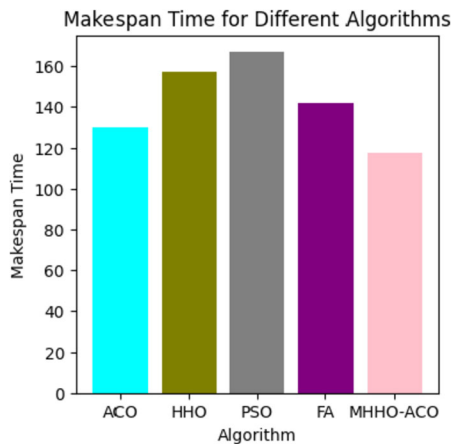


Fig. 8 Comparison of makespan time in a Partial-fault Scenario

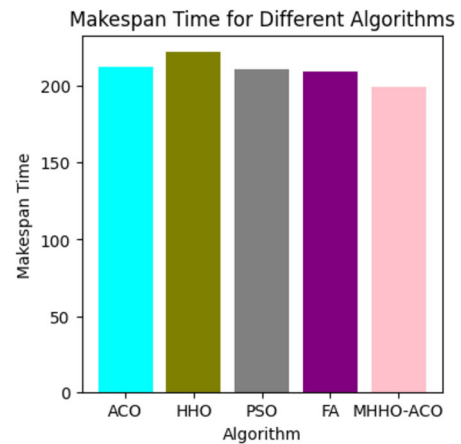


Fig. 11 Comparison of makespan time in a Full-Fault Scenario

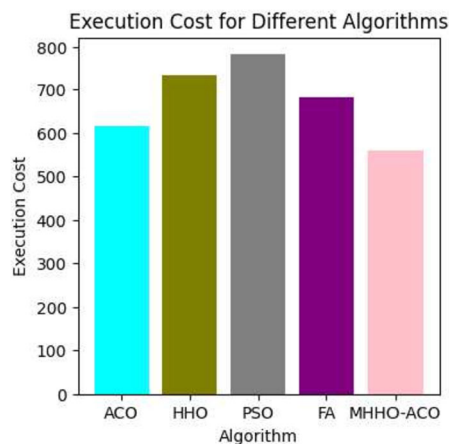


Fig. 9 Comparison of execution cost in a Partial-fault Scenario

security. The performance of this solution surpasses that of conventional load balancing strategies and fog computing approaches that are not optimised.

The results of makespan time, execution cost, and energy consumption of the suggested hybrid meta-heuristic algorithm were compared to those of four different optimization algorithms (HHO, ACO, PSO, and FA). The findings revealed that the MHHO-ACO hybrid algorithm is far better than the competition in each of the three categories shown in Figs. 5, 6, 7, 8, 9, 10, 11, 12, 13. The final comparison graphs are shown in Figures 14, 15, 16. In these graphs, the metrics are assessed for the entire failure scenario, which includes 500 IoT devices, 25 fog nodes, and 5000 tasks. Here, MHHO-ACO consistently demonstrates its ability to manage a variety of defect scenarios, cost-effectiveness, energy efficiency, high availability,

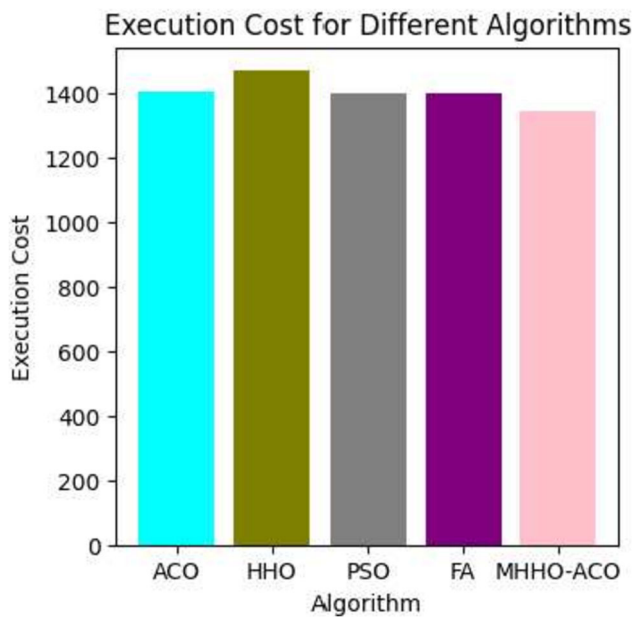


Fig. 12 Comparison of execution time in a Full-Fault Scenario

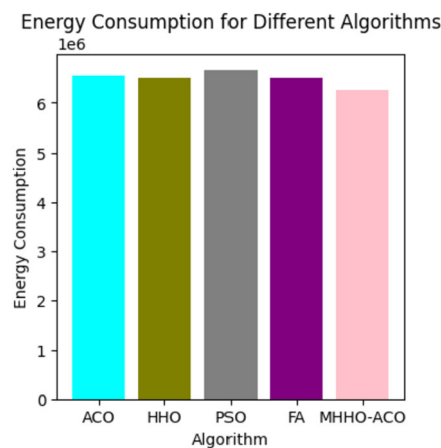


Fig. 13 Comparison of energy consumption in a Full-Fault Scenario

scalability, low overhead, and strong security compared to other algorithms. The MHHO-ACO hybrid algorithm has the shortest makespan time of all the methods, at 92.22. This indicates that the task might be finished using the suggested method in the smallest period of time, which is a desired result for fog computing systems. The suggested algorithm's execution cost was 470.21, which was once again the lowest of all algorithms. This suggests that the suggested algorithm is more economical than the alternatives. Last but not least, the suggested algorithm had the lowest energy usage of all methods at 116.01. This implies that the proposed approach consumes a smaller amount of energy than the other methods.

The outcomes of availability, scalability, overhead, and security are depicted in Table 6. Compared to previous

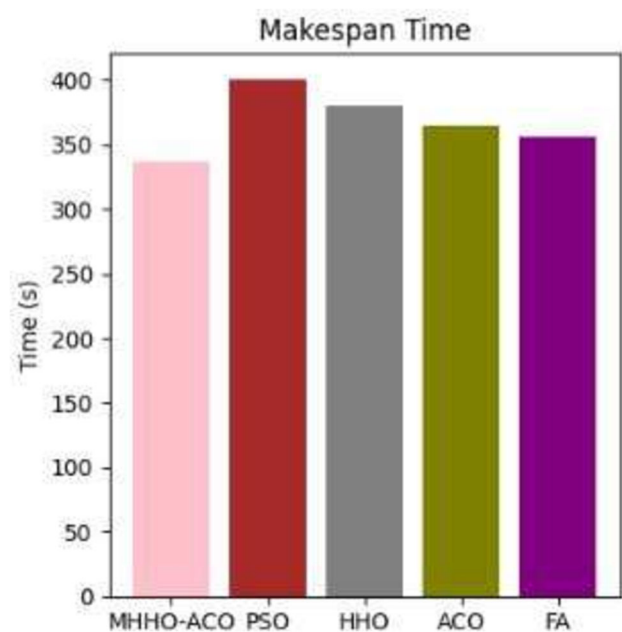


Fig. 14 Comparison of makespan time with different algorithms

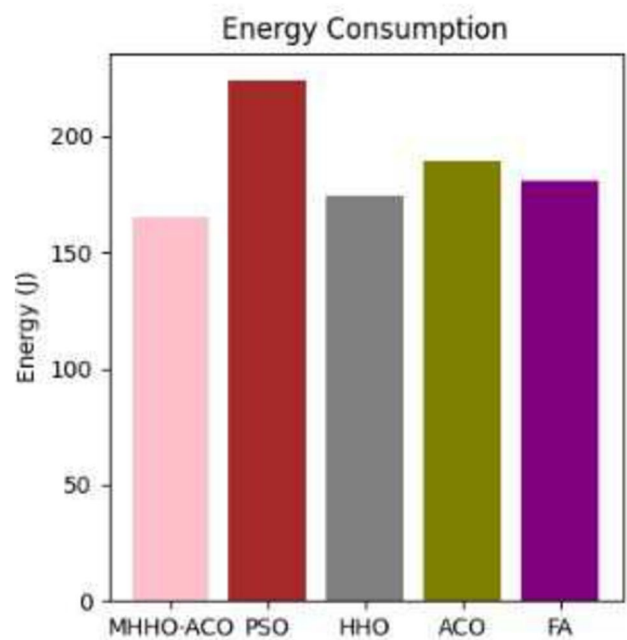


Fig. 15 Comparison of energy consumption with different algorithms

algorithms, it demonstrates that the suggested approach offers good availability, scalability, and security with low overhead. It means the proposed hybrid algorithm combining MHHO and ACO is effective in achieving fault-tolerant load balancing in fog computing. The MHHO component enables the algorithm to manage dynamic workload changes and alter load distribution in real-time, while the ACO component assists the algorithm in rapidly and efficiently finding optimal solutions. The suggested

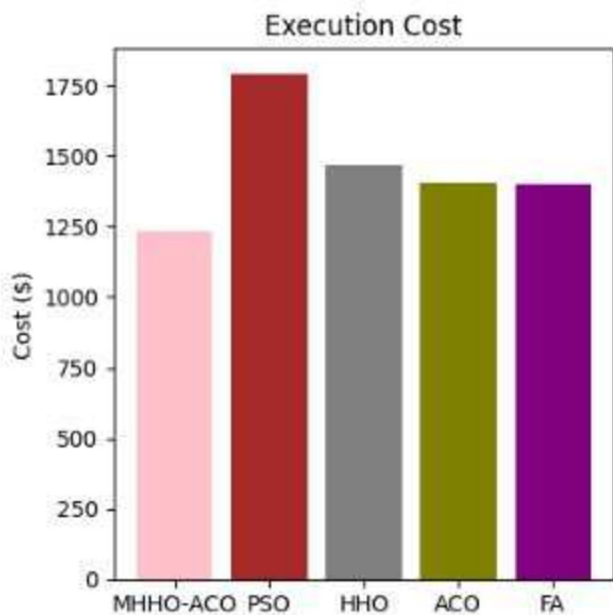


Fig. 16 Comparison of execution cost with different algorithms

technique achieves fault tolerance using a redundancy-based strategy in which many instances of the same application are deployed on separate fog nodes, and in the event of a node failure, the burden is diverted to the remaining nodes. This maintains service continuity and decreases the chance of service disruption.

The findings show that the suggested hybrid algorithm is a potential approach for fault-tolerant load balancing in fog computing systems.

7 Conclusion and future scope

A hybrid meta-heuristic method combining MHHO and ACO is created and shown in this paper to achieve fault tolerance-aware load balancing in fog computing. The proposed method considers the dependability and energy efficiency of the fog nodes, as well as network traffic and task characteristics, to create a balanced workload distribution that reduces makespan time, execution cost, and energy consumption.

The results of the experiments showed that the suggested MHHO-ACO algorithm did better than the other meta-heuristic algorithms (HHO, ACO, PSO, and FA) in terms of makespan time, execution cost, and energy use. In comparison to the next best-performing algorithm, the algorithm obtained a 19% reduction in makespan time, a 10% reduction in execution cost, and a 25% reduction in energy usage. Furthermore, it has minimal overhead, great

availability, scalability, and security. This illustrates the suggested approach's efficacy and efficiency in establishing fault tolerance-aware load balancing in fog computing.

Future research might look into incorporating other fault tolerance techniques, such as Byzantine fault tolerance and redundancy-based approaches, to improve the system's dependability and resilience. Furthermore, the suggested technique may be expanded in real-time to manage dynamic workloads and node failures, making it more appropriate for realistic applications in fog computing.

Author contributions VK: Introduction, Organization, Literature Rivew, Proposed Method, Evaluation. RA: Background, Comparison study with matrices and table, AK: Partial evaluation with review and editing, Result and Discussion with Conclusion. VK, RA, AK read and approved the final manuscript.

Funding No funding was received for conducting this study.

Data availability Data availability is not applicable to this article.

Declarations

Conflict of interest We confirm that this work is original and has not been published elsewhere, nor is it currently under consideration for publication elsewhere. We have no conflicts of interest to disclose. This article does not contain any studies with animals performed by any of the authors. This article does not contain any studies with human participants or animals performed by any of the authors.

References

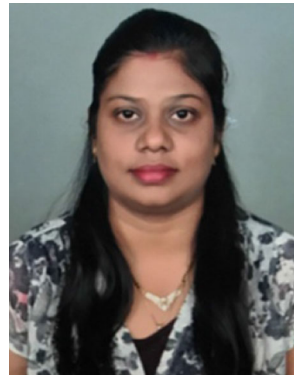
1. Ningning, S., Chao, G., Xingshuo, A., Qiang, Z.: Fog computing dynamic load balancing mechanism based on graph repartitioning. *China Commun.* **13**(3), 156–164 (2016)
2. Rathore, N.K., Khan, Y., Kumar, S., Singh, P., Varma, S.: An evolutionary algorithmic framework cloud based evidence collection architecture. *Multimed. Tools Appl.* **3**, 1–29 (2023)
3. Verma, M., Bhardwaj, N., Yadav, A.K.: Real time efficient scheduling algorithm for load balancing in fog computing environment. *Int. J. Inf. Technol. Comput. Sci* **8**(4), 1–10 (2016)
4. Khattar, N., Sidhu, J., Singh, J.: Toward energy-efficient cloud computing: a survey of dynamic power management and heuristics-based optimization techniques. *J. Supercomput.* **75**, 4750–4810 (2019)
5. Liu, L., Chang, Z., Guo, X., Mao, S., Ristaniemi, T.: Multiobjective optimization for computation offloading in fog computing. *IEEE Internet Things J.* **5**(1), 283–294 (2017)
6. Wan, J., Chen, B., Wang, S., Xia, M., Li, D., Liu, C.: Fog computing for energy-aware load balancing and scheduling in smart factory. *IEEE Trans. Ind. Inform.* **14**(10), 4548–4556 (2018)
7. Saini, P., Ahuja, R.: A review for predicting the diabetes mellitus using different techniques and methods. In: *Proceedings of International Conference on Data Science and Applications: ICDSA 2021*, Volume 1, pp. 425–440 (2022). Springer

8. Neto, E.C.P., Callou, G., Aires, F.: An algorithm to optimise the load distribution of fog environments. In: 2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 1292–1297 (2017). IEEE
9. Gupta, K., Gupta, D., Kukreja, V., Kaushik, V.: Fog computing and its security challenges. In: Machine Learning for Edge Computing, pp. 1–24. CRC Press, Boca Raton (2022)
10. Rathore, N., Chana, I.: Variable threshold-based hierarchical load balancing technique in grid. *Eng. Comput.* **31**(3), 597–615 (2015)
11. Singh, S.P., Nayyar, A., Kaur, H., Singla, A.: Dynamic task scheduling using balanced vm allocation policy for fog computing platforms. *Scalable Comput.* **20**(2), 433–456 (2019)
12. Rathore, N., Chana, I.: Load balancing and job migration techniques in grid: a survey of recent trends. *Wirel. Personal Commun.* **79**(3), 2089–2125 (2014)
13. Rathore, N.K., Chana, I.: A cognitive analysis of load balancing technique with job migration in grid environment. In: World Congress on Information and Communication Technology (WICT), Mumbai, In: IEEE Proceedings Paper, pp. 77–82 (2011)
14. Mounnan, O., El Mouatasim, A., Manad, O., Hidar, T., Abou El Kalam, A., Idboufker, N.: Privacy-aware and authentication based on blockchain with for iot enabled fog computing. In: 2020 Fifth International Conference on Fog and Mobile Edge Computing (FMEC), pp. 347–352 (2020). IEEE
15. Hussein, M.K., Mousa, M.H.: Efficient task offloading for iot-based applications in fog computing using ant colony optimization. *IEEE Access* **8**, 37191–37201 (2020)
16. Yakubu, I.Z., Murali, M.: An efficient meta-heuristic resource allocation with load balancing in iot-fog-cloud computing environment. *J. Ambient Intell. Hum. Comput.* **12**, 1 (2023)
17. Haris, M., Zubair, S.: Mantaray modified multi-objective harris hawk optimization algorithm expedites optimal load balancing in cloud computing. *J. King Saud Univ.-Comput. Inform. Sci.* **34**(10), 9696–9709 (2022)
18. Nishant, K., Sharma, P., Krishna, V., Gupta, C., Singh, K.P., Nitin, Rastogi, R.: Load balancing of nodes in cloud using ant colony optimization. In: 2012 UKSim 14th International Conference on Computer Modelling and Simulation, pp. 3–8 (2012). <https://doi.org/10.1109/UKSim.2012.11>
19. Sim, K.M., Sun, W.H.: Ant colony optimization for routing and load-balancing: survey and new directions. *IEEE Trans. Syst. Man Cybern. Part A* **33**(5), 560–572 (2003). <https://doi.org/10.1109/TSMCA.2003.817391>
20. Kumar, A., Kumar, R., Sharma, A.: Energy aware resource allocation for clouds using two level ant colony optimization. *Comput. Inform.* **1**, 37 (2018)
21. Kishor, A., Chakrabarty, C.: Task offloading in fog computing for using smart ant colony optimization. *Wirel. Pers. Commun.* **22**, 1 (2021)
22. Kaur, M., Aron, R.: Focalb: Fog computing architecture of load balancing for scientific workflow applications. *J. Grid Comput.* **19**(4), 40 (2021)
23. Fan, Q., Ansari, N.: Towards workload balancing in fog computing empowered iot. *IEEE Trans. Netw. Sci. Eng.* **7**(1), 253–262 (2018)
24. Baek, J.-y., Kaddoum, G., Garg, S., Kaur, K., Gravel, V.: Managing fog networks using reinforcement learning based load balancing algorithm. In: 2019 IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–7 (2019). IEEE
25. Xu, G., Pang, J., Fu, X.: A load balancing model based on cloud partitioning for the public cloud. *Tsinghua Sci. Technol.* **18**(1), 34–39 (2013)
26. Rathore, N., Chana, I.: A sender initiate based hierarchical load balancing technique for grid using variable threshold value. In: 2013 IEEE International Conference on Signal Processing, Computing and Control (ISPCC), pp. 1–6 (2013). IEEE
27. Rathore, N., Chana, I.: Report on hierarchal load balancing technique in grid environment. *i-manager's J. Inform. Technol.* **2**(4), 21 (2013)
28. Islam, M.S.U., Kumar, A.: A context-aware priority tuple scheduling for fog computing paradigm. *Trans. Emerg. Telecommun. Technol.* **89**, 4647 (2022)
29. Chandak, A., Ray, N.K.: A review of load balancing in fog computing. In: 2019 International Conference on Information Technology (ICIT), pp. 460–465 (2019). IEEE
30. Kaur, N., Kumar, A., Kumar, R.: Promo: proactive mobility-support model for task scheduling in fog computing. *Int. J. Comput. Appl.* **44**(11), 1092–1101 (2022)
31. Alarifi, A., Abdelsamie, F., Amoon, M.: A fault-tolerant aware scheduling method for fog-cloud environments. *PLoS ONE* **14**(10), 0223902 (2019)
32. Rathore, N.: Performance of hybrid load balancing algorithm in distributed web server system. *Wirel. Person. Commun.* **101**(3), 1233–1246 (2018)
33. Sharif, A., Nickray, M., Shahidinejad, A.: Fault-tolerant with load balancing scheduling in a fog-based iot application. *IET Commun.* **14**(16), 2646–2657 (2020)
34. Zhang, X., Rane, K.P., Kakaravada, I., Shabaz, M.: Research on vibration monitoring and fault diagnosis of rotating machinery based on internet of things technology. *Nonlinear Eng.* **10**(1), 245–254 (2021)
35. Singh, A., Moser, L.E., Melliar-Smith, P.: Integrating fault tolerance and load balancing in distributed systems based on Corba. In: European Dependable Computing Conference, pp. 154–166 (2005). Springer
36. Laxkar, P., Rathore, N.K.: Load balancing algorithm in distributed network. *Solid State Technol.* **89**, 6633–6645 (2020)
37. Rathore, N.: An enhancement of gridsim architecture with load balancing. *PROCEEDINGS BOOK* (2021)
38. Wang, K., Shao, Y., Xie, L., Wu, J., Guo, S.: Adaptive and fault-tolerant data processing in healthcare iot based on fog computing. *IEEE Trans. Netw. Sci. Eng.* **7**(1), 263–273 (2018)
39. Rathore, N.K.: Efficient hierarchical load balancing technique based on grid. In: 29 Th MP Young Scientist Congress, p. 55 (2014)
40. Sharif, A., Nickray, M., Shahidinejad, A.: Fault-tolerant with load balancing scheduling in a fog-based iot application. *IET Commun.* **14**(16), 2646–2657 (2020)
41. Rathore, N.K.: Efficient agent based priority scheduling and loadbalancing using fuzzy logic in grid computing. *System* **6**, 13–23 (2015)
42. Abdel-Basset, M., Abdel-Fatah, L., Sangaiah, A.K.: Meta-heuristic algorithms: A comprehensive review. In: Computational Intelligence for Multimedia Big Data on the Cloud with Engineering Applications, pp. 185–231. Elsevier, Amsterdam (2018)
43. Cho, K.-M., Tsai, P.-W., Tsai, C.-W., Yang, C.-S.: A hybrid meta-heuristic algorithm for vm scheduling with load balancing in cloud computing. *Neural Comput Appl.* **26**(6), 1297–1309 (2015)
44. Abdel-Basset, M., El-shahat, D., Elhoseny, M., Song, H.: Energy-aware metaheuristic algorithm for industrial internet of things task scheduling problems in fog computing applications. *IEEE Internet Things J.* **8**(16), 12638–12649 (2020)
45. Khattak, H.A., Arshad, H., Islam, S., Ahmed, G., Jabbar, S., Sharif, A.M., Khalid, S.: Utilization and load balancing in fog servers for health applications. *EURASIP J. Wirel. Commun. Netw.* **2019**(1), 91 (2019)

46. Gokul, M., Balamurali, M.: Cloud load balancing using meta-heuristics. In: 2022 6th International Conference on Intelligent Computing and Control Systems (ICICCS), pp. 589–595 (2022). IEEE
47. Oma, R., Nakamura, S., Duolikun, D., Enokido, T., Takizawa, M.: Fault-tolerant fog computing models in the iot. In: International Conference on P2P, Parallel, Grid, Cloud and Internet Computing, pp. 14–25 (2018). Springer
48. Mohamed, N., Al-Jaroodi, J., Jawhar, I.: Towards fault tolerant fog computing for iot-based smart city applications. In: 2019 IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC), pp. 0752–0757 (2019). IEEE
49. Rajab, H.T., Younis, M.F.: Dynamic fault tolerance aware scheduling for healthcare system on fog computing. Iraqi J. Sci. **89**, 308–318 (2021)
50. Ramzanpoor, Y., Hosseini Shirvani, M., Golsorkhtabaramiri, M.: Multi-objective fault-tolerant optimization algorithm for deployment of iot applications on fog computing infrastructure. Complex Intell. Syst. **8**(1), 361–392 (2022)
51. Singh, S.P.: Effective load balancing strategy using fuzzy golden eagle optimization in fog computing environment. Sustain. Comput. **35**, 100766 (2022)
52. Rathore, N.: Dynamic threshold based load balancing algorithms. Wirel. Person. Commun. **91**(1), 151–185 (2016)
53. Rathore, N., Chana, I.: Job migration with fault tolerance based qos scheduling using hash table functionality in social grid computing. J. Intell. Fuzzy Syst. **27**(6), 2821–2833 (2014)
54. Abuhamdah, A., Al-Shabi, M.: Hybrid load balancing algorithm for fog computing environment. Int. J. Softw. Eng. Comput. Syst. **8**(1), 11–21 (2022)
55. Sumathi, M., Vijayaraj, N., Raja, S.P., Rajkamal, M.: Hho-aco hybridized load balancing technique in cloud computing. Int. J. Inform. Technol. **89**, 1–9 (2023)
56. Annie Poornima Princess, G., Radhamani, A.: A hybrid meta-heuristic for optimal load balancing in cloud computing. J. Grid Comput. **19**(2), 21 (2021)
57. George, S.S., Pramila, R.S.: An efficient load balancing technique using caviar-hho enabled vm migration and replica management in cloud computing. In: Web Intelligence, pp. 1–21. IOS Press
58. Montazerolghaem, A., Khosravi, M., Rezaee, F., Khayyambashi, M.R.: An optimal workflow scheduling method in cloud-fog computing using three-objective harris-hawks algorithm. In: 2022 12th International Conference on Computer and Knowledge Engineering (ICCKE), pp. 300–306 (2022). IEEE
59. Hassan, K., Javaid, N., Zafar, F., Rehman, S., Zahid, M., Rasheed, S.: A cloud fog based framework for efficient resource allocation using firefly algorithm. In: Advances on Broadband and Wireless Computing, Communication and Applications: Proceedings of the 13th International Conference on Broadband and Wireless Computing, Communication and Applications (BWCCA-2018), pp. 431–443 (2019). Springer
60. Baburao, D., Pavankumar, T., Prabhu, C.: Load balancing in the fog nodes using particle swarm optimization-based enhanced dynamic resource allocation method. Appl. Nanosci. **13**(2), 1–10 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Vijaita Kashyap has completed her M.Tech in Software System from S.A.T.I, Vidisha (M.P). She is currently pursuing her Ph.D. Program from Chitkara University, Punjab, and working as an Assistant Professor in the Department of Computer Science and Engineering. She has 11 years of teaching experience in computer science dept. Her area of interest includes Cloud Computing, Fog Computing, and IoT. She has published 10 research papers in Conferences, SCI and Scopus Journals. She has also filed 4 design and 2 utility patents.



Dr. Rakesh Ahuja secured Ph.D. degree in the field of Computer Science & Engineering. He is having experience of 26 years of experience in Academics, Research, Administration, and Industries. He is currently working as a Professor in the Department of Computer Science & Engineering, at Chitkara University, Punjab, India. He has published more than fifty papers in international journals, and reputed conferences. He also filed 27 national and international patents. His research area includes Machine learning, cloud computing, Digital Right Management, Multimedia Security, Pattern recognition, and Information Hiding. His research interest is in the development schemes of information, and multimedia security for and database management systems. He has filed more than 25 Patents. Currently, he is working on solving problems of healthcare sector through machine learning and deep learning technologies.



Dr. Ashok Kumar is currently working as a Professor in Lovely Professional University, Punjab, India. He is heading School of Computer Application. He is Ph.D. in Computer Science and Engineering from Thapar University, Punjab, India. He did Master of Technology (M. Tech.) in Computer Science and Engineering from Punjabi University, Patiala, India. He has 20+ years of teaching and research experience. He has filed 20 patents, out of that four patents have been granted. He has published more than 50 research articles in various International Journals and Conferences of repute. He has also published four edited books. Under his supervision, 2 Ph.D. theses have been awarded and 7 Ph.D. theses are on-going. He also has several Master's theses to his credit. His current areas of research interest include Cloud/Fog/Edge Computing, Internet of Things, and Machine learning.